

経路探索の 実装と提出

仕様を、動くコードと検証にする

情報システム学科 3年 / 選択2単位

今日の到達目標

- 重みなしグラフで最少辺数の経路を実装できる
- 正常系と境界ケースを、仕様に対応させて検証できる
- 自分のコードとAIの支援範囲を説明して提出できる

授業内で実装・確認・提出を行う。正式な提出先・期限は授業時の案内に従う。

前回との接続

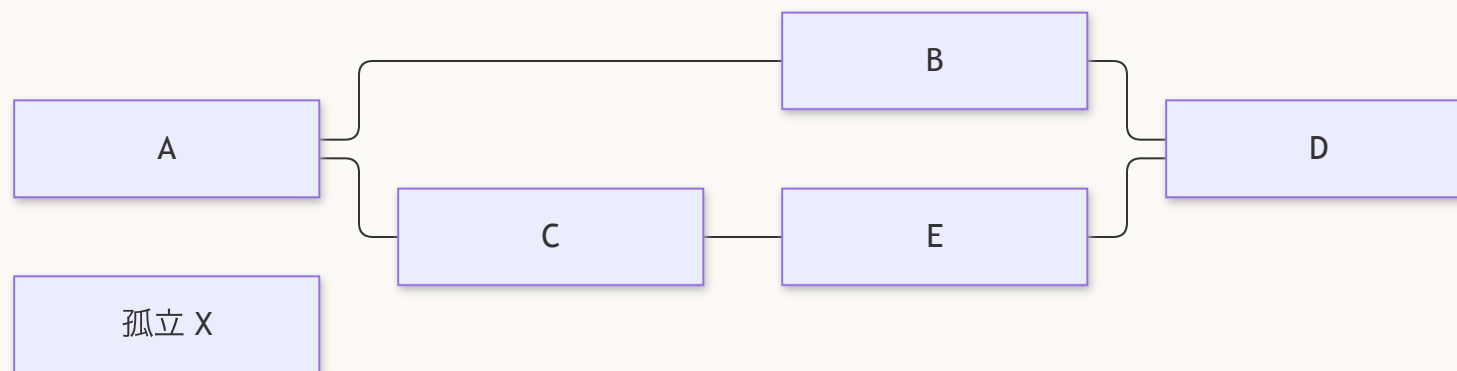
前は、頂点を発見したときに親を記録した。

今回は、その記録を使い、出発点から到着点までの経路を返す。

- キュー：次に処理する頂点
- 親の辞書：発見済みの判定と復元の方法
- 関数の戻り値：経路、または到達不能

題材と前提

架空の駅を結ぶ **無向・重みなしグラフ** を使う。



各辺は移動1回。同じ路線・駅でも、運行時間を表すモデルではない。

入力と出力

出発駅と到着駅を、コマンドライン引数で指定する。

```
python3 examples/06-route-project/route_starter.py A D
```

最終的に期待する経路は **A, B, D**、移動数は **2**。

引数を省略するとAからDを使う。入力データは `route_data.py` にある。

条件ごとの期待値

入力	経路関数の期待値
AからD	<code>['A', 'B', 'D']</code>
AからA	<code>['A']</code>
AからB	<code>['A', 'B']</code>
AからX	<code>None</code>

未知の駅名は入力エラー。CLIは到達不能・入力エラーとも終了コード2で伝える。

学生版の開始状態

`route_starter.py` は、同じ駅と隣の駅だけ进行处理する。

```
python3 examples/06-route-project/route_starter.py A A
python3 examples/06-route-project/route_starter.py A B
python3 examples/06-route-project/route_starter.py A D
```

AからDの多段経路は未実装。最初の失敗は、実装する範囲を示す。

`find_route` のTODOを中心に、自分の作業コピーで実装する。

作業コピーの準備

Python 3.12以上の環境で、リポジトリのルートから学生版とデータを同じ作業先へコピーする。

```
mkdir -p work/06  
cp examples/06-route-project/route_starter.py work/06/route.py  
cp examples/06-route-project/route_data.py work/06/route_data.py  
python3 work/06/route.py A D
```

以後は `work/06/route.py` を編集する。

段階1：仕様の記録

実装前に `work/06/README.md` へ書く。

- 最短とは何を最少にすることか
- 未知の駅と、存在する孤立駅の違い
- 同じ駅を入力したときの経路と移動数
- 経路が複数あるとき、何を保証するか

期待値を先に書くと、出力に合わせて仕様を変える誤りを避けやすい。

段階2：探索の初期状態

開始点だけをキューへ入れ、親を持たない開始点を記録する。

```
queue = deque([start])  
parent = {start: None}
```

未発見の頂点だけを追加する。開始点も最初から発見済み。

未知の頂点の検証は、探索の前に行う。

段階3：次の頂点の発見

```
node = queue.popleft()
for neighbor in graph[node]:
    if neighbor not in parent:
        parent[neighbor] = node
        queue.append(neighbor)
```

`while queue:` の中に置く処理の中心部分。

取り出しには `popleft()` を使い、先に発見した頂点から処理する。

段階4：到着と終了

到着点を発見または取り出したら、経路復元へ進める。

- 探索を最後まで行っても、到着点がなければ `None`
- 到着点が発見済みなら、親をたどって復元
- キューが空になることが、到達不能時の終了条件

どの時点で停止するかは、自分の実装に合わせて説明する。

段階5：経路の復元

親をたどる向きは、到着点から出発点への逆向き。

```
path = []
node = goal
while node is not None:
    path.append(node)
    node = parent[node]
return list(reversed(path))
```

辺数は `len(path) - 1`。同じ駅の経路も、この式で0になる。

正常系の実行

```
python3 work/06/route.py A D  
python3 work/06/route.py D A  
python3 work/06/route.py A A
```

期待する経路：A B D、D B A、A。

それぞれの移動数：2、2、0。出力だけでなく、各隣接駅が辺で結ばれていることも確認する。

境界ケースの実行

```
python3 work/06/route.py A X  
python3 work/06/route.py UNKNOWN D
```

到達不能と未知の入力を、それぞれ説明できる表示にする。

このCLI契約では、どちらも終了コード2。期待する失敗をテストし、成功時と混同しない。

経路の検証条件

文字列の一致だけでは、経路の正しさを十分に調べられない。

1. 先頭が出発点、末尾が到着点
2. 隣り合う2頂点が元のグラフの辺
3. 最少辺数が既知の小さい例で一致
4. 到達不能・未知の入力を仕様どおり扱う

自作入力も1件追加し、その期待値を手で求める。

循環を含む入力

今回の駅グラフには、A B D E C A の循環がある。

訪問済みの判定が働けば、各頂点の発見は1回で済む。

- `parent` を上書きし続けているか
- 同じ頂点を繰り返しキューへ入れていないか
- 親の連鎖が開始点へ戻るか

途中のキューと親の記録で確認する。

失敗の切り分け

症状	先に見る場所
終わらない	訪問済み判定、更新
経路が逆向き	復元後の反転
長い経路になる	FIFO、発見時の親記録
同駅で移動数1	頂点数と辺数の混同

1つの入力に絞って、状態の変化を追う。

AIによる支援の使い方

質問例：「このキューの遷移がBFSになる理由を、入力A Dで説明して」

- 提案を読んで、採用する変更を自分で選ぶ
- 自分の入力で再実行し、期待値と照合する
- 最後は、資料を閉じてでも処理順を説明できるか確認する

AI利用の有無を記録する。未使用の場合も「未使用」と書く。

公開参考版の使い方

`route_reference.py` は、本教材の解説・照合用の参考実装。

自分の実装と検証結果を先に記録し、比較した場合はその範囲も書く。

```
python3 examples/06-route-project/route_reference.py A D
```

参考版のコードを読むことと、自分が理由を説明できることを区別する。 正式な課題での参照条件は、授業時の案内に従う。

提出物の準備

本教材の推奨構成。正式な提出方法は担当教員の案内に従う。

- `route.py` と必要な `route_data.py`
- `README.md` : 実行方法、前提、設計の説明
- `test-results.md` : 入力、期待値、実際の結果
- `reflection.md` : AI支援、検証、残る疑問

提出前に、作業ディレクトリからの実行をもう一度確認する。

理解の確認

1. `parent[start]` を `None` にする理由は何か
2. `pop()` を使うと、最少辺数の保証に何が起こるか
3. `len(path) - 1` が0になる入力を説明できるか
4. 到達不能のときに、経路復元を始めてはいけない理由は何か

次回の質疑に向け、説明に詰まる箇所を振り返りへ記す。

授業内の提出確認

実装、検証、成果物整理が揃ったら、案内された方法で提出する。

提出できたことを確認し、使った版のコードと結果を手元に残す。

未完成の部分がある場合は、動く範囲・再現入力・残る問題を書く。

次回は、提出内容を題材に設計と理解を説明する。

まとめと次回

最少辺数を求める仕様を、キュー・親記録・境界処理として実装した。

検証には、正常な経路と、意図した失敗の両方が必要。

次回は講評と質疑。コードの各行だけでなく、**その選択の理由**を話す。

課題全文：`exercises/06.md`。

参考資料

- [Python公式：collections.deque](#)
- [MIT 6.006：BFSの講義資料](#)

架空の駅データ、学生版、参考版は本教材用の自作。

本教材の到達基準は学習の目安。正式な点数配分は設定していない。